



Instituto Politécnico
de Viana do Castelo

Programação I

Ficheiros Textos

Agenda



- Conceitos básicos sobre stream e ficheiros;
- Tipo de ficheiros;
- Abertura de ficheiros: modos de abertura; função de abertura;
- Manipulação de ficheiros: funções de leitura e escrita;
- Acesso sequencial e direto;
- Fechar ficheiros;
- Outras funções sobre ficheiros.

Stream e ficheiros



- Um stream é uma sequência de dados que pode ser lida ou escrita em um fluxo contínuo, ou seja, em tempo real, enquanto um ficheiro é uma unidade de armazenamento em disco que contém dados que podem ser lidos e escritos permanentemente.
- Um stream é um conceito abstrato que se refere a um fluxo de dados que pode ser lido ou gravado, independentemente da fonte ou destino do fluxo. A linguagem C processa todas as entradas de dados (teclado, rato, etc), saídas de dados (ecrã, impressora, etc) e entrada/saída de dados (discos, disquetes, etc) utilizando streams.
- A grande vantagem em utilizar streams para a entrada/saída de dados é o facto de não ser necessário saber que tipo de periférico está a ser utilizado, dizendo-se que é independente do dispositivo.
- Desta forma, não é necessário escrever códigos diferentes para enviar os dados para uma impressora ou para o ecrã, pois os dados são sempre vistos como conjuntos sequenciais de bytes.



- Os tipos de streams funcionam com ficheiros que são abertos automaticamente na execução de um programa em C:
 - Stdin - representa o “standard input” e está normalmente associado ao teclado.
 - Stdout - representa o “standard output” e está normalmente associado ao ecrã.
 - Stderr - representa o “standard error” que é o local para onde devem ser enviadas as mensagens de erro dos programas.
 - Stdaux - está definido como a porta principal de comunicações, ou seja, a COM1 num PC.
 - Stdprn - representa o “standard printer” e está normalmente associado à impressora.



- Exemplo 1: Leitura de entrada do usuário a partir do stdin e gravação de saída no stdout

```
#include <stdio.h>
```

```
int main() {  
    int num;  
    printf("Digite um número: ");  
    scanf("%d", &num);  
    printf("O número digitado foi: %d\n", num);  
    return 0;  
}
```



- Exemplo 2: Gravação de mensagens de erro no stderr

```
#include <stdio.h>
```

```
int main() {  
    int num;
```

```
    printf("Digite um número: ");  
    if (scanf("%d", &num) != 1) {  
        fprintf(stderr, "Erro: entrada inválida\n");  
        return 1;  
    }
```

```
    printf("O número digitado foi: %d\n", num);
```

```
    return 0;
```

```
}
```



- O uso de stderr é uma **boa prática de programação** em C, especialmente para lidar com erros e exceções durante a execução do programa.
- **Em vez de interromper a execução do programa abruptamente**, exibir mensagens de erro em stderr (em uma barra de mensagens, console ou ficheiro) permite que o programa lide com exceções de forma mais elegante, permitindo que o usuário ou o sistema operacional saiba o que aconteceu de errado.
- Além disso, **a separação entre saída padrão e saída de erro pode ajudar a depurar o programa**, pois é possível redirecionar a saída padrão e a saída de erro para diferentes destinos, como um arquivo de log ou um terminal separado. Isso pode facilitar a localização e o tratamento de erros no código.



- Exemplo 3: Gravação de mensagens de erro no stderr direcionado para um ficheiro

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *arquivo;
```

```
    arquivo = freopen("erros.txt", "w", stderr);
```

```
    int num;
```

```
    printf("Digite um número: ");
```

```
    if (scanf("%d", &num) != 1) {
```

```
        fprintf(stderr, "Erro: entrada inválida\n");
```

```
        return 1;
```

```
    }
```

```
    printf("O número digitado foi: %d\n", num);
```



A função `freopen()` permite redirecionar um fluxo de saída para um ficheiro em vez de escrevê-lo na saída padrão (`stdin`). A sintaxe da função é a seguinte:

```
FILE *freopen(const char *filename, const char *mode, FILE *stream);
```

De modo que...

filename - nome do ficheiro para o qual o fluxo de saída será redirecionado.

mode - a forma como o ficheiro será aberto, como "w" para escrita ou "a" para adicionar conteúdo ao final do ficheiro.

stream - o fluxo de saída que será redirecionado, como `stderr`.



Uma vez que estes ficheiros são abertos automaticamente quando o programa é executado, por exemplo, para escrever uma mensagem na impressora basta fazer:

```
fprintf(stdprn, "Olá Mundo !! \n");
```

Os dados introduzidos pelo teclado não estão associados a um ficheiro, mas é possível processar a leitura desses dados como se viessem de um ficheiro.

Por exemplo, as seguintes instruções são equivalentes:

```
scanf("%d %c", &a, &b);
```

```
fscanf(stdin, "%d %c", &a, &b);
```



Com streams as entradas de dados são feitas a partir do teclado, ou da linha de comando, e todas as saídas de dados são feitas para o monitor por padrão.

No final da execução do programa todos os dados manipulados durante a sua execução são “perdidos”.

Para evitar “perder” esses dados e para aumentarmos as funcionalidades dos programas, vamos aprender armazenar dados em ficheiros e ler os dados de ficheiros.

Em C, um ficheiro é um conjunto de bytes colocados uns a seguir aos outros de forma sequencial.



Tipos de Ficheiros: Na linguagem C existem dois tipos de ficheiros: ficheiros de texto e ficheiros binários

Ficheiro de texto: Um ficheiro é considerado de texto quando é constituído por caracteres perceptíveis para o ser humano: São caracteres da tabela ASCII como algarismos, letras do alfabeto, caracteres de acentuação, pontuação e outros como é o caso do carácter “New Line” que, apesar de não ser visível, representa uma mudança de linha.

Ficheiro Binários: Um ficheiro binário pode ser constituído por qualquer carácter existente na tabela ASCII, podendo conter caracteres especiais. Os ficheiros binários não estão organizados por linhas, são escritos em blocos de memória para o disco e lidos em blocos do disco para a memória.



Sempre que pretendemos trabalhar com ficheiros é necessário:

1. Em primeiro lugar abrir o ficheiro, antes de ler ou escrever de/para o ficheiro.
2. Depois de aberto, podemos realizar todas as operações sobre o ficheiro, como seja: ler dados, escrever dados, fazer o posicionamento, etc.
3. Por fim, depois de efetuar as operações necessárias sobre o ficheiro, é necessário fechar o ficheiro.



Regra geral, na linguagem C, as funções de processamento de ficheiros iniciam-se com a letra f (abreviatura de file).

Em C, para abrir um ficheiro usa-se a função `fopen()`. Esta função devolve um apontador para o ficheiro.

Este apontador, aponta para uma estrutura que contém informação referente ao ficheiro como o nome, a posição atual, se foi aberto para escrita ou leitura, etc.

Este apontador é usado sempre que se acede ao ficheiro, tanto para leituras como para escritas.

Um apontador para ficheiro é uma variável que tem de ser declarada previamente.



Para declararmos um apontador para ficheiro, usa-se o tipo `FILE`, pré-definido da linguagem C, que se encontra na biblioteca `<stdio.h>`.

Pode ser usado da seguinte forma:

```
FILE *nomeApFicheiro=NULL;
```

A função `fopen()` como o próprio nome indica, abre o ficheiro. Se o ficheiro for aberto com sucesso, associa-o a um apontador para ficheiro. Se não conseguir abrir o ficheiro, a função devolve `NULL`. A função tem a seguinte sintaxe:

```
FILE *fopen(char *nome, char *modo)
```

O primeiro argumento da função é uma string com o nome do ficheiro a abrir.

O segundo argumento é uma string com o modo de abertura do ficheiro.

O valor de retorno da função `fopen()` associa o nome físico de um ficheiro à variável do nosso programa. Desta forma, evita-se estar a escrever o nome completo do ficheiro sempre que precisamos de o referenciar.



Os principais modos de abertura de um ficheiro texto são apresentados na tabela:

Modo	Significado
“r”	Abre para leitura. Devolve NULL se o ficheiro não existir ou o utilizador não tiver permissões.
“w”	Abre para escrita. É criado um ficheiro com o nome passado na função. Se já existir, o ficheiro é truncado. Se não conseguir abrir, devolve NULL.
“a”	Abre para adicionar. Se o ficheiro não existir cria-o, se já existir, coloca-se no final de forma a permitir a escrita dos dados a partir dos já existentes.
“r+”	Abre para leitura e escrita (posiciona-se no início do ficheiro)
“w+”	Abre para leitura e escrita (o ficheiro é truncado)
“a+”	Abre para leitura e escrita (posiciona-se no fim do ficheiro)



Resumo dos modos de abertura de um ficheiro texto:

Modo	Leitura?	Escrita?	Se não existe	Se já existe	Posição inicial
r	Sim	Não	NULL	OK	Início
w	Não	Sim	Cria	Recria	Início
a	Não	Sim	Cria	OK	Fim
r+	Sim	Sim	Cria	Permite alterar	Início
w+	Sim	Sim	Cria	Recria	Início
a+	Sim	Sim	Cria	Permite acrescentar	Fim



Exemplo:

Para abrir o ficheiro “teste.txt” para escrita podemos usar:

```
FILE *fp=NULL;  
fp = fopen("teste.txt", "w");
```

Dado que a função pode não conseguir abrir um ficheiro, devolvendo NULL, nesse caso, é necessário verificar o valor de retorno do apontador.

```
FILE *fp=NULL;  
fp = fopen("teste.txt", "w");  
if (fp==NULL) {  
    fprintf(stderr, "Erro: Ocorreu um ERRO ao abrir o ficheiro!\n");  
    exit(-1);  
}
```



Depois de efetuar as operações necessárias sobre o ficheiro, é necessário fechar o ficheiro.

Para retirar a ligação efetuada na abertura dos ficheiros entre uma variável do nosso programa e o ficheiro existente no disco, há necessidade de efetuar o fecho do ficheiro.

Para isso recorre-se à função `fclose()`, cuja sintaxe é a seguinte:

```
int fclose(FILE *fp);
```

Esta função devolve 0, no caso de existir sucesso no fecho do ficheiro, ou a constante EOF no caso de ocorrer um erro.

Antes do ficheiro ser fechado são gravados todos os dados que possam existir em buffers associados ao ficheiro, sendo também libertada a memória alocada pela função `fopen()` para a estrutura do tipo `FILE`.



Exemplo de abertura de um ficheiro para leitura:

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    FILE *fp=NULL;
    char fileName[100];
    puts("Introduza o Nome do Ficheiro:");
    gets(fileName);
    fp = fopen(fileName,"r");
    if (fp==NULL){
        fprintf(stderr, "Erro ao abrir o ficheiro %s\n", fileName);
    }
    else {
        fprintf(stderr, "Ficheiro %s aberto com sucesso!\n", fileName);
        fclose(fp);
    }
}
```



Para escrever informação para ficheiros de texto podemos usar uma das seguintes funções:

```
int fputc(int ch, FILE *fp);
```

```
int putc(int ch, FILE *fp);
```

```
int fputs(const char *s, FILE *fp);
```

```
int fprintf(FILE *fp, const char *format[...]);
```



```
int fputc(int ch, FILE *fp);
```

A função `fputc()` permite escrever um caractere num ficheiro previamente aberto para escrita (modo “w” ou “a”), onde `fp` é o apontador para o ficheiro devolvido pelo `fopen()` e `ch` é o caracter a escrever no ficheiro.

Se a função for bem sucedida devolverá o caracter escrito, se for mal sucedida devolve o caracter `EOF`.

`EOF` é um caracter especial definido na linguagem C que significa fim de ficheiro (End Of File).

A função `putc(int ch, FILE *fp)` funciona da mesma forma que a função `fputc(int ch, FILE *fp)`.



```
int fputs(const char *str, FILE *fp);
```

A função `fputs()` permite escrever uma string num ficheiro previamente aberto para escrita (modo “w” ou “a”), onde `fp` é o apontador para o ficheiro devolvido pelo `fopen()` e `str` é a string a escrever no ficheiro.

Nota: a string não pode conter um caractere “new line”.

Se a função for mal sucedida devolve o caractere `EOF`. Se for bem sucedida devolve um zero.



```
int fprintf( FILE *fp, const char *format[,arg1,..]);
```

Permite a escrita formatada de dados em ficheiros de texto. O modo de utilização desta função é semelhante à congénere `printf()` só que, enquanto esta escreve na saída padrão (monitor), a função `fprintf()` permite escrever para o ficheiro associado ao apontador `fp`, dado como primeiro argumento da função.

Se a função `fprintf` for mal sucedida (se ocorrer um erro na gravação) devolve o caractere `EOF`. Se for bem sucedida devolve o número de bytes gravados.



Exemplo de `fprintf`: O exemplo seguinte cria um ficheiro de texto e escreve para o ficheiro os números inteiro de 0 a 9, um em cada linha, logo após o cabeçalho.

```
int main() {  
    FILE *pf=NULL;  
    int i=0;  
    fp = fopen("exemplo.txt", "w");  
    if (fp == NULL) {  
        printf("Problemas na CRIACAO do ficheiro \n");  
        return -1;  
    }  
    fprintf(fp,"Numeros entre 0 e 9:\n");  
    for (i=0;i<10;i++) {  
        fprintf(fp,"%d\n",i);  
    }  
    fclose(fp);  
}
```



Para ler informação de um ficheiro de texto podemos usar uma das seguintes funções:

```
int fscanf( FILE *fp, const char *format[...]);
```

```
int fgetc(FILE *fp);
```

```
int getc(FILE *fp);
```

```
char *fgets(char *s, int n, FILE *fp);
```



```
int fgetc(FILE *fp)
```

Permite ler um caracter de um ficheiro previamente aberto em modo leitura (“r”).

O argumento é um apontador para o ficheiro devolvido pelo `fopen()`.

A função `fgetc()` devolve EOF quando o fim do ficheiro é atingido.

A função `getc(FILE *fp)` funciona da mesma forma que a função `fgetc()`

Assim, para ler um ficheiro até ao fim poderemos usar o código

```
ch = getc(fp);  
while (ch!=EOF) {  
    ch = getc(fp);  
}
```



Exercício: Escreva um programa que implemente o comando `type` do MS-DOS ou `cat` do Linux, sendo o nome do ficheiro passado na linha de comando.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *fp=NULL;
    int ch;
    if (argc!=2) {
        fprintf(stderr, "Erro nos argumentos !...\n");
        exit(-1);
        /* Termina o Programa */
    }
    fp = fopen(argv[1], "r");
    if (fp==NULL) {
        fprintf(stderr, "Erro ao abrir o ficheiro %s\n", argv[1]);
        return -1;
    }
    while ((ch=fgetc(fp))!=EOF){
        printf("%c", ch);
    }
    fclose(fp);
    return 0;
}
```



```
char *fgets(char *s, int n, FILE *fp);
```

A função `fgets()` permite ler uma string (incluindo o caracter “new line”) de um ficheiro de texto.

O primeiro argumento (`s`) é a string resultante da leitura. O segundo argumento (`n`) é o número máximo de caracteres a ler. O terceiro argumento (`fp`) é o apontador para o ficheiro devolvido pelo `fopen()` aberto em modo de leitura.

A função lê texto até encontrar um “new line” ou até um máximo `n-1` caracteres. A string resultante é terminada pelo caracter nulo ‘\0’.

A função retorna a string lida, caso corra bem. Retorna `NULL` caso chegue ao fim de ficheiro (encontrou o caracter EOF) ou ocorrer um erro.



```
int fscanf(FILE *fp, const char *format[, arg1, ...]);
```

A função `fscanf()` permite ler dados de um ficheiro de texto.

O modo de utilização desta função é semelhante à congénere `scanf()` só que enquanto esta lê da entrada padrão (teclado), a função `fscanf()` permite ler do ficheiro associado ao apontador `fp`, passado como primeiro argumento para a função.

A função `fscanf()` devolve o número de parâmetros que conseguiu ler com sucesso.



Deteção de fim de ficheiro

Já vimos que há funções que devolvem um caracter especial (EOF) sempre que é detetado o fim do ficheiro.

A deteção de fim de ficheiro pode ser efetuada usando a função `feof()` com a seguinte sintaxe:

```
int feof(FILE *fich)
```

A função `feof()` devolve o valor 1 quando é atingido o fim do ficheiro e pode ser usada da seguinte forma: `while(!feof(fp)) {...}`.

A função `feof()` só deteta uma situação de fim de ficheiro depois de ter sido realizada uma operação sobre o ficheiro, sendo que a deteção de fim de ficheiro num ficheiro acabado de abrir devolve Falso, mesmo para um ficheiro vazio.



O Exemplo seguinte lê um ficheiro texto, linha a linha, com um máximo por linha, de 99 caracteres e imprime o seu conteúdo para o ecrã.

```
#include <stdio.h>
int main() {
    FILE *fp=NULL;
    char linha[100], *result=NULL;
    fp = fopen("Teste.txt", "r"); // Abre um ficheiro para LEITURA
    if (fp == NULL) {
        fprintf(stderr, "Erro na abertura do ficheiro\n");
        return -1;
    }
    while (!feof(fp)) { // Lê uma linha com o '\n' inclusive
        result = fgets(linha, 100, fp); // lê até 99 caracteres ou até o '\n'
        if (result) {
            printf("%s", linha);
        }
    }
    fclose(fp);
    return 0;
}
```



O Exemplo seguinte lê um ficheiro texto, linha a linha, soma os seus valores e imprime o resultado para o ecrã. O ficheiro contém um valor inteiro por linha.

```
#include <stdio.h>
int main() {
    FILE *fp=NULL;
    int val=0, soma=0, res=0;
    fp = fopen("valores.txt", "r"); // Abre um ficheiro para LEITURA
    if (fp == NULL) {
        sprintf(stderr, "Problemas na abertura do ficheiro \n");
        return -1;
    }
    while(!feof(fp)){
        res=fscanf(fp,"%i", &val);
        if (res > 0){
            soma += val;
            printf("Valor lido: %i; valor da soma: %i \n", val, soma);
        }
    }
    fclose(fp);
    return 0;
}
```

o teu • de partida



Instituto Politécnico
de Viana do Castelo

www.ipvc.pt